Tutorial_9_Custom_Provider

Tutorial 9: Creating a Custom LLM Provider

Duration: 30 minutes | **Level**: Advanced

Overview

Extend HelixCode with your own LLM provider by implementing the `Provider` interface.

Prerequisites

- Go 1.26.0 or later
- Running HelixCode development environment
- Access to your LLM API

Step 1: Provider Scaffold

Create `internal/llm/providers/myprovider/myprovider.go`:

```
package myprovider

import (
    "context"
    "dev.helix.code/llm"
)

type MyProvider struct {
    apiKey string
    endpoint string
    model   string
}
```

Implement the `llm.Provider` interface: - `Name() string` — return `"myprovider"` - `Generate(ctx context.Context, req *llm.Request) (*llm.Response, error)` - `GenerateStream(ctx context.Context, req *llm.Request) (<-chan *llm.Response, error)`

Step 2: Configuration

Register in `config/config.yaml`:

```
llm:
  providers:
    myprovider:
      type: myprovider
```

```
    endpoint: "https://api.myprovider.com/v1"
    api_key: "${MYPROVIDER_API_KEY}"
    enabled: true
```

Step 3: Testing

```
func TestMyProvider_Generate(t *testing.T) {
    p := &MyProvider{apiKey: os.Getenv("MYPROVIDER_API_KEY")}
    resp, err := p.Generate(context.Background(), &llm.Request{
        Messages: []llm.Message{{Role: "user", Content:
        "Hello"}},
    })
    assert.NoError(t, err)
    assert.NotEmpty(t, resp.Content)
}
```

Step 4: Registration

In `internal/llm/providers/registry.go`:

```
import "dev.helix.code/llm/providers/myprovider"

func init() {
    Register("myprovider", func(cfg Config) (llm.Provider,
        error) {
        return &myprovider.MyProvider{
            apiKey:   cfg.APIKey,
            endpoint: cfg.Endpoint,
            model:    cfg.Model,
        }, nil
    })
}
```

Build: `go build ./...` — should succeed.

Sources verified

Sources verified 2026-05-29: https://go.dev/dl/ (go1.26.3 latest stable Go; 1.24 past support) ; project go.mod (root go 1.25.2, inner go 1.26) + CLAUDE.md §3.1 (PostgreSQL 15+).